

Part XI

Linear Classification

Lecture Outline

- ① Recap and Motivation
- ② Linear Classification for Two Classes
- ③ Learning Algorithms
- ④ Feature Engineering
- ⑤ Probabilistic Approaches
- ⑥ Probabilistic Approaches: Logistic Regression
- ⑦ Multi-Class Logistic Regression

Recap: Linear Regression

- We previously studied **Linear Regression** models where for inputs $x \in \mathbb{R}^d$ and output $y \in \mathbb{R}$, we choose a parametric model of the form:

$$\hat{y} = g(x) = \sum_{m=0}^{M-1} w_m \phi_m(x) = w^\top \phi(x),$$

with a chosen set of M basis functions $\phi(x) = [\phi_0^\top(x) \ \dots \ \phi_{M-1}^\top(x)]^\top$ and weights $w = [w_0 \ \dots \ w_{M-1}]^\top$ to predict continuous outputs (e.g., $y \in \mathbb{R}$). Here, $\hat{y}$ is the estimated output.

- Parameter estimation was done via *Least-Squares* (or equivalently MLE under Gaussian noise).
- **Question:** How can we adapt a similar idea to solve classification problems where outputs are discrete (e.g., $y \in \{0, 1\}$ or $y \in \{-1, 1\}$)?

Motivation for Classification

- In classification, our targets y are labeled (e.g., $\{-1, +1\}$ or $\{1, \dots, K\}$) rather than continuous values.
- We want to construct a discriminant function that separates classes.
- Today, we will see how linear models can also be adapted for classification tasks.

Linear Classification: The Basic Model

- Consider a simple linear model with $\phi_0(x) = 1$ (bias) and $\phi_1(x) = x_1, \dots, \phi_d(x) = x_d$:

$$\hat{y} = w^\top \phi(x) = \bar{w}^\top x + w_0,$$

with $\bar{w} = [w_1, \dots, w_d]^\top$.

- If we know that $y \in \{-1, 1\}$, then this model **will not work**.

Linear Classification: The Basic Model

- Consider a simple linear model with $\phi_0(x) = 1$ (bias) and $\phi_1(x) = x_1, \dots, \phi_d(x) = x_d$:

$$\hat{y} = w^\top \phi(x) = \bar{w}^\top x + w_0,$$

with $\bar{w} = [w_1, \dots, w_d]^\top$.

- If we know that $y \in \{-1, 1\}$, then this model **will not work**.
- Instead, we choose

$$\hat{y} = f(w^\top \phi(x)) = f(\bar{w}^\top x + w_0),$$

where $f(a) = \text{sign}(a)$ (+1 if $a \geq 0$, and -1 if $a < 0$). Here, f is known as the *activation function*.

Linear Classification: The Basic Model

- $\hat{y} = \text{sign}(w^\top \phi(x)) = \text{sign}(\bar{w}^\top x + w_0).$

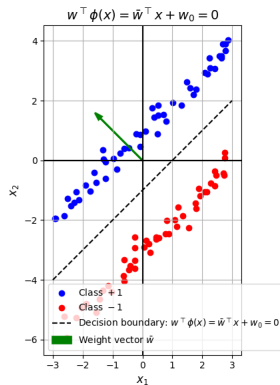
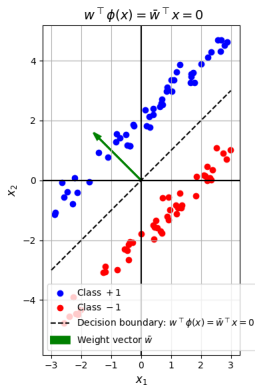


Figure: 2D example (no bias).

Linear Classification: The Basic Model

- $\hat{y} = \text{sign}(w^\top \phi(x)) = \text{sign}(\bar{w}^\top x + w_0)$.
- The weight vector w (also $\bar{w}$) are orthogonal to the decision boundary since $w^\top \phi(x) = 0$ ($\bar{w}^\top (x_A - x_B) = 0$ for any two x_A, x_B on the decision boundary.)

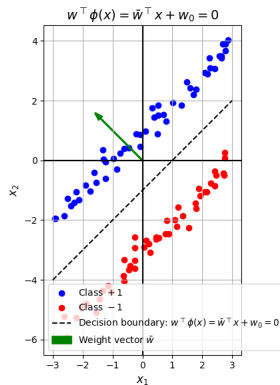
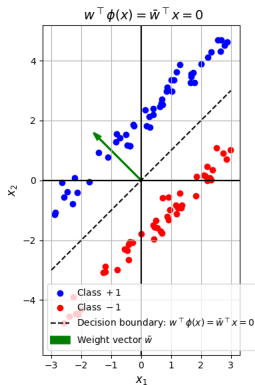


Figure: 2D example.

Linear Classification: The Basic Model

- $\hat{y} = \text{sign}(w^\top \phi(x)) = \text{sign}(\bar{w}^\top x + w_0)$.
- The weight vector w (also $\bar{w}$) are orthogonal to the decision boundary since $w^\top \phi(x) = 0$ ($\bar{w}^\top (x_A - x_B) = 0$ for any two x_A, x_B on the decision boundary.)

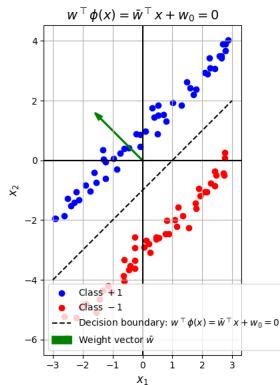
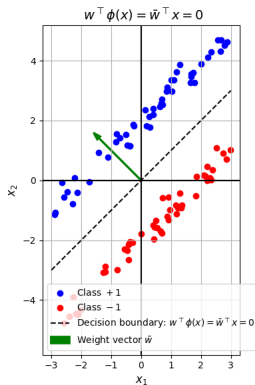


Figure: 2D example.

Multi-Class Linear Classification

- For K different classes (e.g., $y \in \{1, \dots, K\}$), we can define K different linear functions:

$$\hat{y}_c = w_c^\top \phi(x), \text{ for } c \in \{1, \dots, K\}.$$

- **Decision Rule:** $\hat{y} = \operatorname{argmax}_{c \in \{1, \dots, K\}} \hat{y}_c$.

Multi-Class Linear Classification

- For K different classes (e.g., $y \in \{1, \dots, K\}$), we can define K different linear functions:

$$\hat{y}_c = \mathbf{w}_c^\top \phi(x), \text{ for } c \in \{1, \dots, K\}.$$

- **Decision Rule:** $\hat{y} = \operatorname{argmax}_{c \in \{1, \dots, K\}} \hat{y}_c$.
- We can write it more compactly as

$$\hat{y} = \mathbf{w}^\top \phi(x),$$

where $\mathbf{w} = [w_1 \ \dots \ w_K]$.

Multi-Class Linear Classification

- For K different classes (e.g., $y \in \{1, \dots, K\}$), we can define K different linear functions:

$$\hat{y}_c = w_c^\top \phi(x), \text{ for } c \in \{1, \dots, K\}.$$

- Decision Rule:** $\hat{y} = \operatorname{argmax}_{c \in \{1, \dots, K\}} \hat{y}_c$.
- This rule creates piecewise linear decision boundaries.

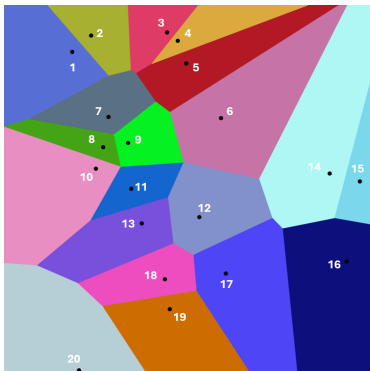


Figure: Voronoi diagram for K -classes.

Multi-Class Linear Classification

- For K different classes (e.g., $y \in \{1, \dots, K\}$), we can define K different linear functions:

$$\hat{y}_c = w_c^\top \phi(x), \text{ for } c \in \{1, \dots, K\}.$$

- Decision Rule:** $\hat{y} = \operatorname{argmax}_{c \in \{1, \dots, K\}} \hat{y}_c$.
- This rule creates piecewise linear decision boundaries.



Aliza Aufrichtig @alizauf · Mar 4

Garlic halved horizontally = nature's Voronoi diagram?

[en.wikipedia.org/wiki/Voronoi_d...](https://en.wikipedia.org/wiki/Voronoi_diagram)



12



234



878



How to find the weights from data?

- Consider that we have N input-output pairs $\{(x_1, y_1), \dots, (x_N, y_N)\}$ with $x_n \in \mathbb{R}^d, y_n \in \{1, \dots, K\}, n \in \{1, \dots, N\}$.
- How can we find the weights for the classifier $\hat{y} = \operatorname{argmax}_{c \in \{1, \dots, K\}} \hat{y}_c$ with $\hat{y} = \mathbf{w}^\top \phi(x)$ to fit the data?

How to find the weights from data?

- Consider that we have N input-output pairs $\{(x_1, y_1), \dots, (x_N, y_N)\}$ with $x_n \in \mathbb{R}^d, y_n \in \{1, \dots, K\}, n \in \{1, \dots, N\}$.
- How can we find the weights for the classifier $\hat{y} = \operatorname{argmax}_{c \in \{1, \dots, K\}} \hat{y}_c$ with $\hat{y} = \mathbf{w}^\top \phi(x)$ to fit the data?
- We will consider the two approaches:
 - Least-Squares (just like our old friend linear regression).
 - The perceptron algorithm.

- **Idea:** Treat each output data $y_n \in \{1, \dots, K\}$ as a K -dimensional vector $\tilde{y}_n$ such that the c -th element of it $\tilde{y}_{n,c} = 1$ if $y_n = c \in \{1, \dots, K\}$ and the rest of the elements are zeros ($\tilde{y}_{n,i \neq c} = 0$).

- **Idea:** Treat each output data $y_n \in \{1, \dots, K\}$ as a K -dimensional vector $\tilde{y}_n$ such that the c -th element of it $\tilde{y}_{n,c} = 1$ if $y_n = c \in \{1, \dots, K\}$ and the rest of the elements are zeros ($\tilde{y}_{n,i \neq c} = 0$). Example: for $y_n \in \{1, 2, 3\}$, if $y_n = 1$, then

$$\tilde{y}_n = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}.$$

- **Idea:** Treat each output data $y_n \in \{1, \dots, K\}$ as a K -dimensional vector $\tilde{y}_n$ such that the c -th element of it $\tilde{y}_{n,c} = 1$ if $y_n = c \in \{1, \dots, K\}$ and the rest of the elements are zeros ($\tilde{y}_{n,i \neq c} = 0$).
- We can then just use the Least-Squares solution for multiple outputs to find $\mathbf{w}$.

$$\mathbf{w}_{\text{LS}} = \underset{\mathbf{w}}{\operatorname{argmin}} \sum_{n=1}^N \frac{1}{2} \|\tilde{\mathbf{y}}_n - \mathbf{w}^\top \phi(x_n)\|^2 = (\Phi^\top \Phi)^{-1} \Phi^\top \tilde{\mathbf{y}}.$$

$$\tilde{\mathbf{y}} = \begin{bmatrix} \tilde{\mathbf{y}}_1^\top \\ \vdots \\ \tilde{\mathbf{y}}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times K}, \text{ and}$$

$$\Phi(\mathbf{x}) = \begin{pmatrix} \phi_0(x_1) & \phi_1(x_1) & \cdots & \phi_M(x_1) \\ \phi_0(x_2) & \phi_1(x_2) & \cdots & \phi_M(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0(x_N) & \phi_1(x_N) & \cdots & \phi_M(x_N) \end{pmatrix}.$$

- We can then just use the Least-Squares solution for multiple outputs to find $\mathbf{w}$.

$$\mathbf{w}_{\text{LS}} = \underset{\mathbf{w} \in \mathbb{R}^{M \times K}}{\operatorname{argmin}} \sum_{n=1}^N \frac{1}{2} \|\tilde{\mathbf{y}}_n - \mathbf{w}^\top \phi(x_n)\|^2 = (\Phi^\top \Phi)^{-1} \Phi^\top \tilde{\mathbf{y}}.$$

- **Weaknesses:**
 - No confidence or probabilistic interpretation.
 - Sensitive to outliers.

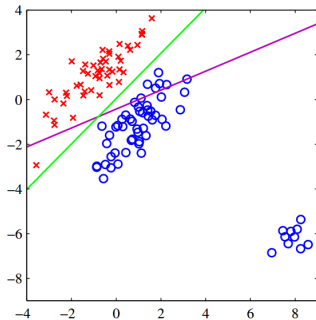
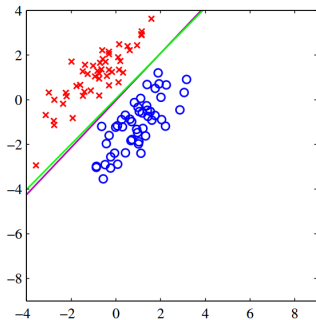


Figure: Example from Bishop.

The Perceptron Algorithm

- Rosenblatt's perceptron (1957), is the *two-class* model

$$\hat{y} = \text{sign}(w^\top \phi(x)),$$

with $y \in \{-1, 1\}$, $x \in \mathbb{R}^d$, $\phi_0(x) = 1, \phi_1(x) = x_1, \dots, \phi_d(x) = x_d$.

The Perceptron Algorithm

- Rosenblatt's perceptron (1957), is the *two-class* model

$$\hat{y} = \text{sign}(w^\top \phi(x)),$$

with $y \in \{-1, 1\}$, $x \in \mathbb{R}^d$, $\phi_0(x) = 1, \phi_1(x) = x_1, \dots, \phi_d(x) = x_d$.

- It is based on an analogy to how biological neurons work.

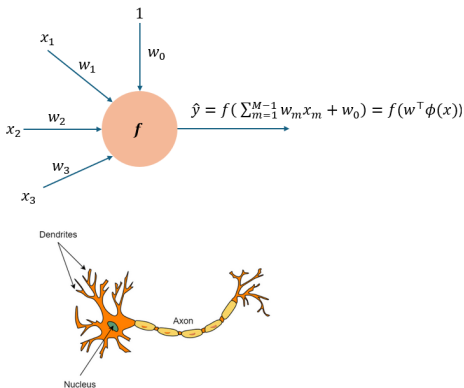


Figure: Neuron and a perceptron.

The Perceptron Algorithm

- Rosenblatt's perceptron (1957), is the *two-class* model

$$\hat{y} = \text{sign}(w^\top \phi(x)),$$

with $y \in \{-1, 1\}$, $x \in \mathbb{R}^d$, $\phi_0(x) = 1, \phi_1(x) = x_1, \dots, \phi_d(x) = x_d$.

- If we have N input-output pairs $\{(x_1, y_1), \dots, (x_N, y_N)\}$, then we want to find the weights w such that $w^\top \phi(x_n) > 0$ if $y_n = 1$ and $w^\top \phi(x_n) < 0$ if $y_n = -1$. This is equivalent to $w^\top \phi(x_n) y_n > 0$.

The Perceptron Algorithm

- Rosenblatt's perceptron (1957), is the *two-class* model

$$\hat{y} = \text{sign}(w^\top \phi(x)),$$

with $y \in \{-1, 1\}$, $x \in \mathbb{R}^d$, $\phi_0(x) = 1, \phi_1(x) = x_1, \dots, \phi_d(x) = x_d$.

- To find the weights $w \in \mathbb{R}^{d+1}$, The perceptron algorithm aims to minimize an error known as the *perceptron criterion*. For each (x_n, y_n) , the perceptron criterion error is

$$E_n^p(w) = \begin{cases} 0, & w^\top \phi(x_n) y_n \geq 0, \\ -w^\top \phi(x_n) y_n, & \text{otherwise.} \end{cases}$$

The Perceptron Algorithm

- Rosenblatt's perceptron (1957), is the *two*-class model

$$\hat{y} = \text{sign}(w^\top \phi(x)),$$

with $y \in \{-1, 1\}$, $x \in \mathbb{R}^d$, $\phi_0(x) = 1, \phi_1(x) = x_1, \dots, \phi_d(x) = x_d$.

- The perceptron algorithm aims to minimize the total error

$$E^p(w) = \sum_{n \in \mathcal{M}} -w^\top \phi(x_n) y_n,$$

where $\mathcal{M}$ is the set of misclassified training data.

The Perceptron Algorithm

- Rosenblatt's perceptron (1957), is the *two-class* model

$$\hat{y} = \text{sign}(w^\top \phi(x)),$$

with $y \in \{-1, 1\}$, $x \in \mathbb{R}^d$, $\phi_0(x) = 1, \phi_1(x) = x_1, \dots, \phi_d(x) = x_d$.

- The perceptron algorithm aims to minimize the total error

$$E^P(w) = \sum_{n \in \mathcal{M}} -w^\top \phi(x_n) y_n,$$

where $\mathcal{M}$ is the set of misclassified training data.

- The algorithm uses the stochastic gradient descent with step size $\eta \in (0, 1)$ and batch size of 1 to update the weights

$$w^{\tau+1} = w^\tau + \eta \phi(x_n) y_n,$$

with τ being the iteration number.

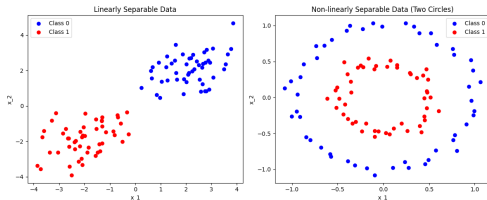
The Perceptron Algorithm

- Rosenblatt's perceptron (1957), is the *two-class* model

$$\hat{y} = \text{sign}(w^\top \phi(x)),$$

with $y \in \{-1, 1\}$, $x \in \mathbb{R}^d$, $\phi_0(x) = 1, \phi_1(x) = x_1, \dots, \phi_d(x) = x_d$.

- The perceptron algorithm is proven to converge only if the data is *linearly-seperable*.



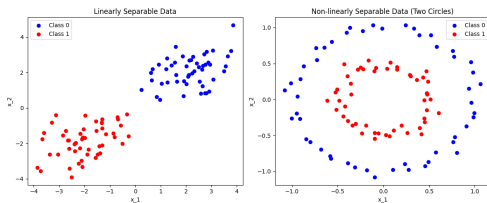
The Perceptron Algorithm

- Rosenblatt's perceptron (1957), is the *two-class* model

$$\hat{y} = \text{sign}(w^\top \phi(x)),$$

with $y \in \{-1, 1\}$, $x \in \mathbb{R}^d$, $\phi_0(x) = 1, \phi_1(x) = x_1, \dots, \phi_d(x) = x_d$.

- The perceptron algorithm is proven to converge only if the data is *linearly-seperable*.



- There is also no simple extension of it to multiple classes.

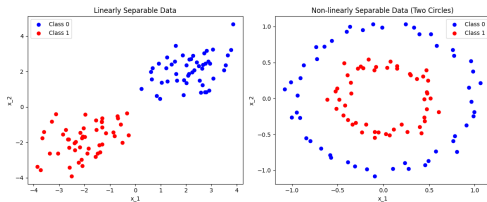
The Perceptron Algorithm

- Rosenblatt's perceptron (1957), is the *two-class* model

$$\hat{y} = \text{sign}(w^\top \phi(x)),$$

with $y \in \{-1, 1\}$, $x \in \mathbb{R}^d$, $\phi_0(x) = 1, \phi_1(x) = x_1, \dots, \phi_d(x) = x_d$.

- The perceptron algorithm is proven to converge only if the data is *linearly-seperable*.



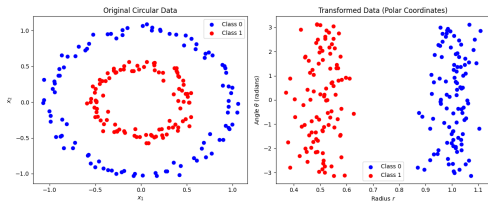
- We also do not have a way to quantify confidence.

Nonlinear Basis Functions

- If we choose the nonlinear basis functions $\phi(x)$ carefully, then we could achieve *linear separability* in the transformed feature space.

Nonlinear Basis Functions

- If we choose the nonlinear basis functions $\phi(x)$ carefully, then we could achieve *linear separability* in the transformed feature space.
- $\phi_1(x) = \sqrt{x_1^2 + x_2^2}$, $\phi_2(x) = \tan^{-1}(\frac{x_2}{x_1})$

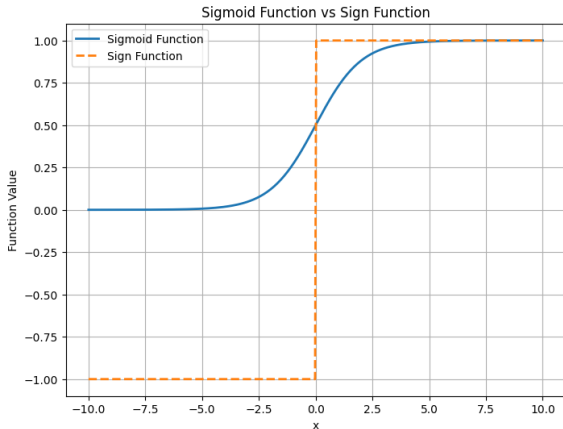


From Sign to Sigmoid

- **Problem:** Sign activation ($f(a) = \text{sign}(a)$) is non-differentiable and lacks probabilistic interpretation.
- **Solution:** Use the logistic sigmoid function for binary classification ($y \in \{0, 1\}$):

$$\sigma(a) = \frac{1}{1 + e^{-a}} \in (0, 1)$$

- Interpret $\sigma(w^\top \phi(x))$ as $p_{Y|X,W}(1 | x, w) = P(Y = 1 | x, w)$.



Logistic Regression Model

- For binary classification ($y \in \{0, 1\}$):

$$p_{Y|X,W}(1 | x, w) = \sigma(w^\top \phi(x)) = \frac{1}{1 + e^{-w^\top \phi(x)}}$$

$$p_{Y|X,W}(0 | x, w) = 1 - \sigma(w^\top \phi(x))$$

Logistic Regression Model

- For binary classification ($y \in \{0, 1\}$):

$$p_{Y|X,W}(1 | x, w) = \sigma(w^\top \phi(x)) = \frac{1}{1 + e^{-w^\top \phi(x)}}$$

$$p_{Y|X,W}(0 | x, w) = 1 - \sigma(w^\top \phi(x))$$

- Can in general be written as

$$p_{Y|X,W}(y | x, w) = \sigma(w^\top \phi(x))^y (1 - \sigma(w^\top \phi(x)))^{1-y}$$

Logistic Regression Model

- For binary classification ($y \in \{0, 1\}$):

$$p_{Y|X,W}(1 | x, w) = \sigma(w^\top \phi(x)) = \frac{1}{1 + e^{-w^\top \phi(x)}}$$

$$p_{Y|X,W}(0 | x, w) = 1 - \sigma(w^\top \phi(x))$$

- **Decision Boundary:** Predict 1 if $\sigma(w^\top \phi(x)) \geq 0.5$ (i.e., $w^\top \phi(x) \geq 0$), else 0 (0-1 loss in bayesian decision theory).
- **Key Advantage:** Provides class probabilities instead of hard labels.

Logistic Regression Model

- For binary classification ($y \in \{0, 1\}$):

$$p_{Y|X,W}(1 | x, w) = \sigma(w^\top \phi(x)) = \frac{1}{1 + e^{-w^\top \phi(x)}}$$

$$p_{Y|X,W}(0 | x, w) = 1 - \sigma(w^\top \phi(x))$$

- If we have N input-output pairs, then we can compactly write it as

$$\boldsymbol{\pi} = \sigma(\Phi(\mathbf{x})\mathbf{w}),$$

$$\text{where } \boldsymbol{\pi} = \begin{bmatrix} p_{Y|X,W}(1 | x_1, w) \\ p_{Y|X,W}(1 | x_2, w) \\ \vdots \\ p_{Y|X,W}(1 | x_n, w) \end{bmatrix}, \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \text{ and}$$

$$\Phi(\mathbf{x}) = \begin{bmatrix} \phi_0(x_1) & \phi_1(x_1) & \cdots & \phi_{M-1}(x_1) \\ \phi_0(x_2) & \phi_1(x_2) & \cdots & \phi_{M-1}(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0(x_N) & \phi_1(x_N) & \cdots & \phi_{M-1}(x_N) \end{bmatrix}.$$

Maximum Likelihood Estimation

- Likelihood for N (independent and identically distributed (iid)) observations:

$$\mathcal{L}(w) = \prod_{n=1}^N p_{Y|X,W}(y_n | x_n, w) = \prod_{n=1}^N \sigma(w^\top \phi(x_n))^{y_n} (1 - \sigma(w^\top \phi(x_n)))^{1-y_n}$$

- Log-likelihood:

$$\ell(w) = \sum_{n=1}^N \left[y_n \ln \sigma(w^\top \phi(x_n)) + (1 - y_n) \ln(1 - \sigma(w^\top \phi(x_n))) \right]$$

- **Cross-Entropy Loss:** $-\ell(w)$ (minimized via gradient descent).

Gradient Descent for Logistic Regression

- Derivative of sigmoid: $\sigma'(a) = \sigma(a)(1 - \sigma(a))$
- Gradient of log-likelihood:

$$\nabla_w \ell(w) = \sum_{n=1}^N \left[y_n - \sigma(w^\top \phi(x_n)) \right] \phi(x_n) = \Phi^\top(\mathbf{x}) (\mathbf{y} - \sigma(\Phi(\mathbf{x})w)),$$

with $\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$.

- Weight update rule:

$$w^{\tau+1} = w^\tau + \eta \sum_{n=1}^N \left[y_n - \sigma(w^\top \phi(x_n)) \right] \phi(x_n) = w^\tau + \eta \Phi^\top(\mathbf{x}) (\mathbf{y} - \sigma(\Phi(\mathbf{x})w))$$

Softmax Activation

- Generalizes sigmoid to K classes. For $c \in \{1, \dots, K\}$:

$$p_{Y|X, \mathbf{w}}(c \mid x, \mathbf{w}) = P(Y = c \mid X = x, \mathbf{w}) = \frac{e^{\mathbf{w}_c^\top \phi(x)}}{\sum_{k=1}^K e^{\mathbf{w}_k^\top \phi(x)}}$$

- **Properties:**
 - Outputs a probability distribution over classes.
- Decision rule: $\hat{y} = \operatorname{argmax}_{c \in \{1, \dots, K\}} p_{Y|X, \mathbf{w}}(c \mid x, \mathbf{w})$

Cross-Entropy Loss for Multi-Class

- One-hot encode labels: $\tilde{y}_n \in \mathbb{R}^K$ (1 at true class, 0 elsewhere).
- Negative log-likelihood (cross-entropy):

$$-\ell(\mathbf{w}) = -\sum_{n=1}^N \sum_{c=1}^K \tilde{y}_{n,c} \ln p_{Y|X, \mathbf{w}}(c \mid x_n, \mathbf{w})$$

- Optimized via gradient descent.

Connection to Gaussian Naive Bayes

- Under Gaussian Naive Bayes with shared variance σ^2 across classes:

$$\ln \frac{P(Y = 1 \mid X = x, w)}{P(Y = 0 \mid X = x, w)} = w^\top x + w_0$$

where $w = \frac{\mu_1 - \mu_0}{\sigma^2}$ and w_0 depends on the priors.

- Logistic regression learns w directly without assuming Gaussianity.
- **Key Insight:** If Naive Bayes assumptions hold, logistic regression weights correspond to log-odds ratios from Naive Bayes.
- See section 4.2 in [Bishop].